

Quantum ISD (2)

<https://youtu.be/QdaYmt1guR4>

QISD

- ISD 알고리즘은 코드기반암호에 대한 가장 강력한 공격 기법
 - 공개키 내 행렬 H_{n-k} (Information set,)을 선택해가며 전수 조사
 - Information set의 역행렬 $\times$ 신드롬 (암호문, s)의 결과 벡터가 조건 해밍 무게를 만족할 경우, 비밀 값 복구 (e)에 성공

$$H = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} \text{Information set 역행렬} \end{pmatrix}^{-1} \times \begin{pmatrix} s \end{pmatrix} = \left[\text{Weight check} = ? \right]$$

Information set
Information set 역행렬

Brute-force

QISD

- 오라클은 3단계로 구성됨
 - Information set, Gauss-Jordan Elimination, Hamming weight

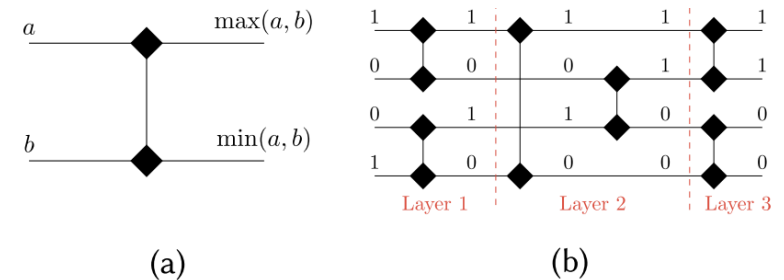
QISD	Module	
Input Setting	Public Key: $H_{(n-k) \times n}$	기존 방식 사용[1]
	Dicke State: $ D_k^n\rangle$	
Oracle	Information set H_{n-k}	[2]를 개선
	Gauss-Jordan Elimination: $(H_{n-k})^{-1}s$	
	Weight $(H_{n-k})^{-1}s$	

[1] Perriello, S., Barenghi, A., & Pelosi, G. (2023). Improving the efficiency of quantum circuits for information set decoding. *ACM Transactions on Quantum Computing*, 4(4), 1-40.

[2] Jang, K., Oh, Y., & Seo, H. (2024). Depth-optimized quantum circuit of Gauss–Jordan elimination. *Applied Sciences*, 14(19), 8579.

Information Set (H_k) 구축

- 기존 방식 그대로 사용하지만, 비용식 다시 계산
 - 본문에 적힌 내용과 회로 구현과 Table 1에 나오는 식의 일부가 다름
 - 재구현해서 비용 추정해본 결과 덤스는 맞는데 게이트 수가 다름 (논문이 더 높게 써져있음)
 - 해당 모듈에서 comparator (a)가 필요하고, 여러 레벨 (b)에 대해 수행됨
 - E.g., 4비트에 대해서, Comparator는 6개 (=2*3) 필요하지만 제시된 수식은 그 이상이 쓰일 때 비용같음



- 재구현 결과와 논문 내용을 토대로 아래와 같은 비용 모델 도출

$$\text{Qubit} \quad n * k + n + \frac{n \log_2(n)(\log_2(n) + 1)}{4}$$

$$\text{Td} \quad 2 \log_2(n)(\log_2(n) + 1) + 4k$$

$$\text{CNOT} \quad \left(\frac{n \log_2(n)(\log_2(n) + 1)}{4} \right) (8k + 9)$$

$$\text{FD} \quad \frac{13}{2} \log_2(n)(\log_2(n) + 1) + 8k$$

$$\text{T} \quad \left(\frac{n \log_2(n)(\log_2(n) + 1)}{4} \right) (7k + 7)$$

Gauss-Jordan Elimination

- 연산량과 큐비트를 줄이도록 변경

현재 pivot이 row2, col2인 상태에 대한 예시

H_k	Before									After								
	col0	col1	col2	col3	col4	col5	col6	col7	c	col0	col1	col2	col3	col4	col5	col6	col7	c
row0	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
row1	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
row2	.	.	P	.	.	.	.	.	.	.	.	P	.	.	.	.	.	.
row3	L	L	.	R	R	R	R	R	C	.	.	.	R	R	R	R	R	C
row4	L	L	.	R	R	R	R	R	C	.	.	.	R	R	R	R	R	C
row5	L	L	.	R	R	R	R	R	C	.	.	.	R	R	R	R	R	C
row6	L	L	.	R	R	R	R	R	C	.	.	.	R	R	R	R	R	C
row7	L	L	.	R	R	R	R	R	C	.	.	.	R	R	R	R	R	C

P: pivot L: lower-left updates R: lower-right updates C: c updates .: Not updated

기존 방식

- Pivot (P)을 기준으로 왼쪽 아래/오른쪽 아래를 모두 업데이트
- L은 다음 단계 연산에는 필요 없지만 c 갱신에 영향을 주는 요소
- c를 해 벡터로 만들기 위한 업데이트 과정이 병렬성이 높음
 - H_k 와 c 사용하여 c 갱신 (Toffoli)

Gauss-Jordan Elimination

- 이전 방식에서 연산량과 큐비트를 줄이도록 변경

현재 pivot이 row2, col2인 상태에 대한 예시

	col0	col1	col2	col3	col4	col5	col6	col7	c
row0	.	.	.	.	.	.	.	.	.
row1	.	.	.	.	.	.	.	.	.
row2	.	.	P	.	.	.	.	.	.
row3	L	L	.	R	R	R	R	R	C
row4	L	L	.	R	R	R	R	R	C
row5	L	L	.	R	R	R	R	R	C
row6	L	L	.	R	R	R	R	R	C
row7	L	L	.	R	R	R	R	R	C

H_k	col0	col1	col2	col3	col4	col5	col6	col7	c
row0	.	.	.	.	.	.	.	.	.
row1	.	.	.	.	.	.	.	.	.
row2	.	.	P	.	.	.	.	.	.
row3	.	.	.	R	R	R	R	R	C
row4	.	.	.	R	R	R	R	R	C
row5	.	.	.	R	R	R	R	R	C
row6	.	.	.	R	R	R	R	R	C
row7	.	.	.	R	R	R	R	R	C

P: pivot L: lower-left updates R: lower-right updates C: c updates .: Not updated

개선 방식

- 오른쪽 아래 (R)만 갱신 (다음 pivot 연산에 필요한 부분만)
- c를 해 벡터로 만들기 위한 과정이 L 생략으로 인해 변경됨
- 이로 인해 마지막 c 보정 과정이 추가

효과

- 연산 수 감소: 매 pivot에서 L 갱신 > C 보정 (반복 연산 vs. 일회성 연산)
- 큐비트 감소: L에 대한 안실라 불필요
- 덱스 증가 : c 보정 연산

소거 단계 전체에 대한 리소스 비교

• T-count (#T)

- 12%~18% 감소 (k 커질수록 개선을 증가) → (참고) 수식 기반으로 개선을 극한 계산하면 약 20% 감소까지 가능

$$T_{old} = \frac{7k(k-1)(5k+8)}{6} \quad T_{new} = \frac{7k(2k^2+3k-5)}{3}$$

$$\Rightarrow \text{Improvement Rate} = \frac{k-2}{5k+8} \times 100\% \Rightarrow \lim_{k \rightarrow \infty} \frac{k-2}{5k+8} = \frac{1}{5} = 20\%$$

• T-depth (Td)

- 6%~20% 증가 (k 커질수록 증가율 감소) → 계산 상 약 1% 증가까지 개선 가능 (소거가 연산 대부분을 차지, 뎁스 증가 원인인 c 보정 단계는 소규모라서 그런듯)

$$Td_{old} = 2k^2 + 2k - 4 \quad (\text{작년 발표자료 기준})$$

$$Td_{new} = 2k^2 + 6k - 8$$

Matrix Size	#CNOT	#1qCliff	#T	Toffoli Depth (TD)	T-Depth *	#Qubit (M)	Full Depth	TD × M	FD × M
$n = 8$	3861	492	3136	35	140	247	407	8645	100,529
$n = 16$	31,329	3352	24,640	135	540	1067	1880	144,045	2,005,960
$n = 32$	251,321	24,368	194,432	527	2108	4435	12,260	2,337,245	54,373,100

• Qubit count (인풋 큐비트 H 포함)

- 16%~21% 감소 (k 커질수록 개선을 증가) → 계산 상 약 22% 감소까지 가능

$$(7k^2 - 5k + 4) / 2$$

이전 버전

k	CNOT	T	Qubit	Td	D	FD	FD-M
8	3360	2744	206	168	254	443	91258
16	25980	20720	858	600	1669	1973	1692834
32	202740	159712	3506	2232	12068	12482	43761892

• Full-depth (FD)

- 2%~9% 증가 (k 커질수록 증가율 감소) → 계산 상 약 1% 증가까지 가능

$$\frac{349}{1024}k^3 + \frac{5}{16}k^2 + \frac{497}{16}k$$

• FD-M

- 9%~20% 감소 (k 커질수록 개선을 증가)

현재 버전 실제 리소스 (자원 비용 식 아님)

Hamming weight

- 결과 벡터가 10110101 인 경우, $1+0+1+1+\dots+1$ 과 같이 모든 비트를 더해야함
- 기존 방식은 $\log(k)$ 레이어의 덧셈 트리가 필요하며, 총 $n-1$ 번의 덧셈 필요
- 각 비트를 한 번에 더하는 멀티 오퍼랜드 덧셈 (QCSA) 으로 변경**
 - 1번으로 해결 + 낮은 텀스의 덧셈기
 - 큐비트는 많이 필요하지만, 큐빗 최대 병목인 소거에서 사용한 안실라로 전체 커버 가능
- 현재 방식이 수식으로 표현하기 깔끔하지 않아서 코드로 대체
 - 기존 방식
 - #T: $4\lceil\log_2(n/2)\rceil^2$
 - Td: $10.5n - 7\lceil\log_2(n/2)\rceil^2$
 - 현재 방식
 - #T: 증가하지만 오라클 전체 관점에서 거의 영향 없음
 - Td: 감소하지만 사실 전체 관점에서 큰 차이는 아님 (굳이 따지자면 Td 감소량이 더 큼)
 - 큰 의미는 없음**

k	T	Td
8	16	56
16	36	105
32	64	224
48	100	329
64	100	497
128	144	1092

기존 방식 ([1], 작년 발표자료 수식 기준)

k	T	Td
8	40	5
16	88	7
32	160	9
64	312	11
128	584	12

현재 버전 실제 리소스 (자원 비용 식 아님)

Q & A